

S-OS — Product Specification

Seedorf Obeng-Mireku

Contents

S-OS — Product Specification & Implementation Plan	1
1. Requirements summary	1
2. Interpretation — what S-OS actually is	2
3. Weak points and unnecessary complexity	3
4. Product concept — final	4
5. Scope by release	5
6. Architecture	6
7. Implementation roadmap	8
8. User journeys — acceptance criteria	9
9. Standing principles	10
10. Immediate next steps	10

S-OS — Product Specification & Implementation Plan

Seedorf Operating System Version 1.0 of this document · 10 August 2026 Author: technical partner notes for Seedorf Obeng-Mireku

1. Requirements summary

1.1 Who this is for

Owner	Seedorf Obeng-Mireku
Title	Software Engineer
Location	Sydney, NSW — open to remote, relocation, visa sponsorship
Education	BSc Computer Science, Western Sydney University — expected Jan 2027Diploma in ICT, WSU International College
Target roles	Software engineering internships and new-grad positions
Experience	No industry internships yet — university and personal projects only
Interests	Interactive software, application security, legal/compliance side of technology

1.2 Confirmed product decisions

Decision	Choice
Visual direction	Nostalgic, not literal. Aero DNA reinterpreted with modern type, spacing and motion
Wallpaper / brand	Direction C — Brand Form. A luminous “S” that serves as wallpaper, boot logo, Start button, favicon and login mark
Authentication	Simulated only. No real accounts, no session backend
Content management	Typed data files committed to git, plus a local-only S-OS Project Installer that generates them from a form
Contact backend	One Next.js route handler + Resend, with honeypot and rate limiting
Mobile	A dedicated S-OS Mobile shell, not a shrunk desktop

Decision	Choice
Sound	Original sounds, muted by default , controlled from the system tray
Unfinished projects	Honest status badges — Stable / Beta / In Development / Planned
Live demos	Per-project embeddable flag: in-window iframe where possible, new tab otherwise
Resume	Placeholder in V1, refined before launch
Pace	Milestone by milestone, no deadline. Valuable now, extensible later

1.3 Project inventory

Project	Status	Notes
S-OS	In development	The portfolio itself. Treated as a first-class installed program
Capstone Management System (WSU)	Details pending	Manages students in professional experience placements and their projects. IP/publishing permission unconfirmed
Date Generator	~75%	AI-assisted date-idea generator for couples. Stack pending
Hotel Management System	Unfinished	Admin daily operations + guest bookings. Stack pending
Farm Management System	Not started	Poultry management

1.4 Open items — blocking full content build

1. **Capstone** — folder not yet connected. The entry is a placeholder with a publication note, and stays that way until the real repository is available.
2. ~~Date Generator~~ — resolved as *Hidden Truths*, written from the repository and deliberately unpublished: it contains private personal material, so it carries a **publicationNote** and no links.
3. ~~Hotel System~~ — resolved as *734*, a Next.js 14 design prototype on mock data.
4. ~~GitHub, LinkedIn, public email~~ — done.
5. ~~Skill levels~~ — reviewed against the repositories in August 2026. Every claim above Learning has a project behind it. Governance and compliance left the registry and stayed in the bio as an interest, where it does not invite a technical question with no project behind it.
6. ~~Security positioning~~ — resolved: application security is evidenced by ADRAH Farms; governance is an interest, not a claimed skill.
7. **Screenshots and demo recordings** — the largest remaining gap. Every gallery is empty and no project declares a demo, so the Screenshots and Demo tabs are hidden everywhere. ADRAH Farms and 734 are the two with real interfaces worth capturing; Nine-Board is the first candidate for a recording, and later for the WebAssembly demo variant its own roadmap lists.
8. **ICT diploma scan** — declared as a document with no file behind it.
9. ~~S-OS specification PDF~~ — done, generated from this file by `npm run docs:spec`.

Dates for 734, Library Management and Nine-Board are taken from repository history rather than from memory. For the two that began as coursework the true start is earlier, but a precise date anyone can check beats an approximate one nobody can.

2. Interpretation — what S-OS actually is

S-OS is not a portfolio with an operating-system theme. It is a **single-page desktop environment implemented in the browser**, whose contents happen to be a professional profile.

The distinction matters technically. A themed portfolio fakes the OS with static pages styled to look like windows. S-OS implements the primitives for real:

- a **window manager** with independent lifecycle, focus, z-order, drag and resize per window

- an **application registry** where programs declare themselves and are mounted on demand
- a **virtual filesystem** that is a real tree structure, traversed identically by File Explorer, My Computer, Start-menu search and the terminal's `cd` and `ls`
- a **command interpreter** parsing input against a registered command table
- a **persistence layer** restoring session state across visits

Those five systems are the actual portfolio. The projects displayed inside them are supporting evidence.

This produces a useful design rule that resolves most future arguments:

One data source feeds every navigation surface.

Adding a project to the registry makes it appear simultaneously on the desktop, in All Programs, in Explorer, in search results, and as a terminal command — with no additional code. If a feature requires content to be duplicated, the feature is designed wrong.

2.1 Positioning

For a candidate with no industry experience, the strategic function of S-OS is to answer “*can this person build non-trivial software?*” before the résumé is read. A window manager written from scratch answers that more convincingly than a fourth CRUD application. S-OS therefore ships as **S-OS.exe**, an installed program with its own architecture write-up, screenshots of the component tree, and an honest engineering-decisions log.

The second differentiator is the security-and-compliance interest. Most junior frontend candidates in Sydney present as React developers. A developer who reasons about privacy, licensing and governance is measurably rarer. Pending confirmation, this becomes a visible pillar rather than a footnote.

3. Weak points and unnecessary complexity

Honest assessment of the original brief. Every item here is a recommendation to cut, defer, or change.

3.1 Critical — must be addressed

W1. Nothing in the OS is crawlable. The brief never mentions SEO, and this is the single largest omission. A recruiter who Googles “Seedorf Obeng-Mireku” must find this site, and the site must have indexable text. A JavaScript desktop shell with client-only content is close to invisible to crawlers and produces a useless link preview when pasted into LinkedIn or an application form.

Fix: Recruiter Mode and every project exist as **real server-rendered routes** — `/recruiter`, `/projects/hotel-manager` — with proper metadata, Open Graph images and structured data. The desktop shell reads the same content client-side. Two presentations, one source. This also gives us deep links.

W2. No deep linking. A single-page desktop means every visitor lands in the same place. You cannot send a hiring manager straight to one project.

Fix: URL is derived from window state. Opening the Hotel project pushes `?app=hotel-manager`; loading that URL boots straight into the desktop with that window open. Shareable, back-button correct, and analytics-legible.

W3. The empty-shelf problem. Covered in the interview. The OS metaphor amplifies whatever it contains. Four thin projects inside an elaborate shell reads as *more* hollow than four thin projects on a plain page. Mitigated by honest status badges, the S-OS.exe entry, and genuine build-log content for incomplete work — but the real fix is finishing the Date Generator and Hotel System before launch.

W4. Boot sequence as a repeated tax. An impressive animation is a cost paid on every visit by someone with limited patience.

Fix: hard ceiling of **2.5 seconds**. Clicking or pressing any key skips immediately. `localStorage` marks the boot as seen and returning visitors get a ~600ms Quick Boot. `prefers-reduced-motion` bypasses it entirely. Any `?app=` or `/recruiter` deep link skips the boot outright — someone arriving from a job application should not sit through a startup animation.

W5. Using genuine Windows 7 assets. Microsoft’s sounds, icons, wallpapers and the Start orb are copyrighted. Shipping them on a portfolio you send to employers is a bad look, particularly for someone positioning around the legal side of software. Everything is original or properly licensed, and the licence provenance of every asset is documented in the repo.

3.2 Overscoped for V1 — defer

W6. S-Browser. A fake browser is a large amount of chrome that reinforces the illusion but carries no professional content. *Merge it with the demo viewer* — when a project is embeddable, it opens inside a minimal browser frame with an address bar showing the real URL. That earns the component. A standalone browser app ships in V1.1 at the earliest.

W7. My Computer and File Explorer are the same application. Building both duplicates a tree view, an address bar, a breadcrumb trail and a content pane. My Computer is simply Explorer opened at the root, showing drives.

W8. Window snapping. Drag-to-edge snapping is fiddly, needs its own drag-intent state machine, and almost nobody will try it. V1.1.

W9. Context menus. Genuinely charming, genuinely optional, and a keyboard-accessibility burden if done properly. V1.1.

W10. Settings with wallpaper, transparency, icon size and theme switching. Each toggle multiplies the visual states needing testing. V1 ships one theme, one wallpaper, and Settings contains only what has real utility: sound, reduced motion, and high contrast.

W11. Recycle Bin. Delightful, zero professional value, cheap to add later. V1.1.

3.3 Changes of approach

W12. Percentage skill bars. Already rejected in the brief — correctly. Device-Manager-style tiers, each linked to the projects that evidence them, are both more honest and more informative.

W13. Terminal scope. An overlong command list is unmaintainable and mostly undiscovered. V1 ships ~15 well-built commands with tab completion, command history and an `open <app>` verb that can launch anything in the registry — the depth is in the quality of the interpreter, not the command count.

W14. Easter eggs. Cap at four. `sudo hire-seedorf`, `matrix`, `coffee`, `credits`. Each must remain professional if a hiring manager finds it. Nothing that mocks employers, nothing crude.

W15. Notifications. Maximum two on first boot, then only in response to explicit user action. Unprompted toasts on a portfolio are noise.

4. Product concept — final

4.1 Identity

Element	Definition
Name	S-OS — Seedorf Operating System
Tagline	<i>A developer workstation you can explore</i>
Mark	A single luminous “S” formed from a continuous ribbon — one SVG serving as wallpaper, boot logo, Start button, favicon and login avatar frame
Palette	Deep navy base #0A0D14 → #12212E; cyan-to-azure accent ramp #8EF1FF → #1A86D6; glass surfaces as translucent white at 6–12% with a 1px inner highlight
Type	One geometric sans for UI, one monospace for terminal and technical values. Both self-hosted and subset
Motion	120–220ms, cubic-bezier ease-out. Windows scale from 96% with opacity. Nothing bounces
Voice	System messages are dry and technical with occasional dryness. Never jokey

4.2 The three entry paths

			Recruiter Mode	Professional overview, fast track
Land	Boot	Login	Continue as Guest	Full desktop
			Demo credentials	Full desktop (+ small easter egg)

Deep link (`/recruiter`, `?app=x`) Skips boot and login entirely

Demo credentials are displayed on the login screen itself, pre-filled and visible. Hiding them is a puzzle nobody has agreed to solve.

4.3 Application catalogue

Application	Purpose	V1
Recruiter Mode	Professional fast track. Everything a recruiter needs in one scrollable window	
Projects	Explorer-style project browser by category	
<i>Project apps</i>	One window per project, generated from the registry	
About Seedorf	System-Properties-styled profile	
Skills	Device-Manager-styled capability tree, each skill linked to evidence	
Resume	In-window document viewer with download	
Documents	Explorer at the Documents node	
Terminal	Interactive command interpreter	
Contact	Form + links, real delivery via Resend	
System Information	S-OS specs, installed technologies, project count	
Computer	Explorer at root, portfolio areas as drives	
Settings	Sound, reduced motion, high contrast only	
Demo Viewer	Minimal browser frame for embeddable project demos	
Recycle Bin	Easter egg	V1.1
S-Browser	Standalone browser	V1.1+

4.4 Drive layout

Computer	
Local Disk (C:)	System - About, System Information, Settings
Projects (D:)	By category: Full-Stack · University · Security · Experiments
Experience (E:)	Education, work, certifications
Skills (F:)	Capability tree
Documents (G:)	Resume, certificates, case studies, architecture docs

The same tree backs Explorer navigation, My Computer, terminal `cd/ls`, and search indexing.

5. Scope by release

Version 1 — the shippable portfolio

Brand system · boot · login · desktop · window manager (open, close, minimise, maximise, restore, focus, drag, resize) · taskbar · Start menu · All Programs · search · Explorer · project applications · Recruiter Mode · About · Skills · Resume viewer · Documents · System Information · Terminal · Contact with working delivery · Demo Viewer · Settings (three toggles) · S-OS Mobile shell · sound muted by default · localStorage persistence · Project Installer (dev-only) · full accessibility pass · SEO routes and deep links · deployed on Vercel.

Version 1.1 — polish

Window snapping · desktop and icon context menus · Recycle Bin · theme switcher (S-OS Dark, Developer Mode) · wallpaper switching · richer notifications · standalone S-Browser · Explorer view modes (icon/list/details) · terminal command expansion · project case-study long-form pages.

Version 2 — the admin release

Database-backed content (Postgres/Supabase) behind the existing data-layer interface · real authentication for the owner account only · **S-OS Admin** in-OS project management with image upload · privacy-respecting analytics · achievement system for exploration · dynamic “S-OS Update” changelog app · interactive embedded project demos.

6. Architecture

6.1 Stack

Layer	Choice	Rationale
Framework	Next.js (App Router)	Server-rendered SEO routes alongside the client shell; one route handler for contact; Vercel deployment
Language	TypeScript, strict: true noUncheckedIndexedAccess on	The content model <i>is</i> the validation.
Styling	Tailwind CSS with CSS custom properties for theme tokens	Tokens in CSS variables make V1.1 theming a variable swap, not a refactor
State	Zustand	See 6.2
Motion	Motion (motion/react)	Layout animations and a first-class useReducedMotion
Icons	Lucide for UI chrome, custom SVG for program icons	Program icons must be original; UI chrome need not be
Email	Resend via route handler	No secrets client-side
Testing	Vitest + Testing Library , Playwright for journeys	Window manager logic is pure and highly testable

No component library. Every window, button and panel is custom — that is the point of the project.

6.2 Why Zustand, not Context

Window dragging updates position state at up to 60fps. React Context re-renders *every* consumer on *every* value change, so a drag on one window would re-render all windows, the taskbar and the desktop, sixty times a second. Zustand subscribes with selectors: a drag re-renders only the window being dragged. Context is still used for genuinely static values — theme tokens, feature flags — where re-render cost is nil.

Stores are split by concern so subscriptions stay narrow:

```
useWindowStore      windows[], focus order, z-index, geometry
useShellStore       Start menu, search, context menus, notifications
useSystemStore      boot phase, session, sound, reduced motion, theme
usePreferencesStore persisted slice → localStorage
```

6.3 Directory structure

```
src/
  app/
    page.tsx           Desktop shell (client)
    recruiter/page.tsx Server-rendered, indexable
    projects/[slug]/page.tsx Server-rendered, indexable
    api/contact/route.ts
   .opengraph-image.tsx
  os/                  The operating system
    boot/              BootManager, BootScreen, LoginScreen
    shell/              Desktop, DesktopGrid, Taskbar, StartMenu, SystemTray
    window/             WindowManager, OSWindow, chrome, drag & resize hooks
    mobile/             MobileShell, AppGrid, Dock, LockScreen
    notifications/
    registry/           applications.ts - the app registry
  apps/                One folder per application
```

recruiter/	projects/	about/	skills/	resume/
documents/	terminal/	contact/	system-info/	
explorer/	settings/	demo-viewer/		
data/				Content - the only place facts live
projects/				One file per project + index
profile.ts	skills.ts	experience.ts	education.ts	
filesystem.ts				Virtual FS derived from the above
lib/				
content/				getProjects(), getProfile() - the V2 swap seam
search/				Index builder and query
terminal/				Command table, parser, executor
sound/				
stores/				
components/ui/				Buttons, panels, tabs, glass surfaces
types/				
styles/tokens.css				
tools/installer/				Dev-only Project Installer

6.4 Content model

```

type ProjectStatus = 'stable' | 'beta' | 'in-development' | 'planned';
type SkillLevel = 'experienced' | 'proficient' | 'working-knowledge' | 'learning';

interface Project {
  id: string; // 'hotel-manager'
  displayName: string; // 'Hotel Management System'
  executable: string; // 'HotelManager.exe'
  icon: string;
  version: string;
  status: ProjectStatus;
  category: ProjectCategory;
  featured: boolean;
  desktopShortcut: boolean;

  tagline: string; // one line, used in search and cards
  overview: string;
  problem: string;
  solution: string;
  role: string; // explicit - recruiters ask what YOU built
  team?: { size: number; yourContribution: string };

  technologies: Technology[];
  features: string[];
  architecture?: string;
  challenges: { challenge: string; solution: string }[];
  lessons: string[];
  buildLog?: { done: string[]; next: string[] }; // for unfinished work

  screenshots: Screenshot[];
  video?: string;
  links: { github?: string; live?: string; docs?: string };
  embeddable: boolean; // may it run inside a Demo Viewer window?

  dateStarted: string;
  dateCompleted?: string;
}

```

Every consumer — desktop icons, Start menu, Explorer, search index, terminal, Recruiter Mode, the server-rendered `/projects/[slug]` route — reads this through `lib/content`. Nothing imports the data files directly. When V2 replaces files with a database, only `lib/content` changes.

6.5 Window manager design

Pure logic, no DOM assumptions, fully unit-testable:

```
interface WindowInstance {
  id: string;
  appId: string;
  title: string;
  icon: string;
  state: 'normal' | 'minimised' | 'maximised';
  bounds: { x: number; y: number; w: number; h: number };
  restoreBounds: Rect | null;
  zIndex: number;
  isFocused: boolean;
  constraints: { minW: number; minH: number; resizable: boolean };
}
```

Notable decisions:

- **z-index from an ordered stack, not arbitrary numbers.** Focus moves the id to the top of an array; z-index is derived from its position. No number inflation, no collisions.
- **Drag/resize via pointer events and transform**, committing to store state on release. Transform during drag avoids layout thrash; committing on release keeps state clean.
- **Applications receive no window chrome.** An app component knows nothing about being in a window — which is precisely why the same components render full-screen in the mobile shell.
- **Lazy mounting.** Apps are `next/dynamic` imports resolved on first launch. The terminal, demo viewer and project apps never load for a visitor who does not open them.

6.6 Accessibility

Not a final-milestone bolt-on; it is a constraint on every component.

- Windows are `role="dialog"` with `aria-labelledby` on the title bar; focus moves into a window on open and returns to the launcher on close; `Escape` closes the focused window
- Full keyboard operation: `Tab` through desktop icons, `Enter` to launch, arrow keys to move focus in grids, `Alt+Tab`-equivalent window cycling, `Ctrl+Esc` for Start
- Windows are draggable **and** resizable by keyboard via a move/resize mode — this is where most OS-metaphor portfolios fail WCAG
- `prefers-reduced-motion` disables boot animation, window transitions and decorative motion
- Every glass surface tested for 4.5:1 text contrast; a high-contrast mode replaces translucency with solid fills
- Semantic HTML inside applications — a project window is `<article>` with real headings, not nested divs
- The server-rendered routes are the screen-reader-friendly path, linked from a skip link on the desktop

6.7 Performance budget

Metric	Target
LCP (desktop, cable)	< 1.5s
Total JS on first load	< 180KB gzipped
Boot-to-desktop	2.5s, skippable
Interaction latency during drag	< 16ms per frame

Enforced by route-level code splitting, dynamic app imports, `next/image` with explicit dimensions for all screenshots, self-hosted subset fonts, sound files lazy-loaded only after unmute, and no animation of properties outside `transform/opacity`.

7. Implementation roadmap

Each milestone ends in a working, reviewable state. Nothing is left half-wired between milestones.

#	Milestone	Delivers	Depends on
0	Foundation	Next.js + TS strict + Tailwind + ESLint/Prettier, folder structure, design tokens, base UI primitives, Vitest running	—
1	Content model & data layer	All types, <code>lib/content</code> accessors, placeholder profile/skills/projects data, virtual filesystem tree	0
2	Brand system	S mark, wallpaper, boot logo, Start button, program icon set, glass surface components	0
3	Boot & login	Boot sequence with skip + Quick Boot, login screen, three entry paths, session state	1, 2
4	Window manager	Store, <code>OSWindow</code> , drag, resize, minimise/maximise/restore, focus, z-order, keyboard control, unit tests	1
5	Desktop & taskbar	Wallpaper, icon grid, taskbar with running-app states, system tray, clock	4
6	Start menu & All Programs	Menu, program tree, power controls, keyboard navigation	5
7	Core professional apps	Recruiter Mode, About, Skills, Resume viewer, System Information	1, 4
8	Projects	Explorer, project application template, Demo Viewer, real project content	1, 7
9	Terminal	Command table, parser, history, tab completion, <code>open</code> , easter eggs	8
10	Search & notifications	Unified index across projects/apps/skills/docs, Start-menu search, toast system	6, 8
11	Contact	Form with validation and states, route handler, Resend, honeypot + rate limit	4
12	S-OS Mobile	Lock screen, app grid, dock, full-screen app presentation reusing app components	7, 8
13	SEO & deep links	<code>/recruiter</code> and <code>/projects/[slug]</code> server routes, URL-synced window state, metadata, OG images, sitemap	8
14	Project Installer	Dev-only form tool generating data files and placing images	1
15	Accessibility & performance audit	Keyboard pass, screen-reader pass, contrast audit, Lighthouse, bundle analysis, reduced-motion verification	all
16	Sound & final polish	Original sound set, tray control, empty/error/loading states, error boundaries	15
17	Deploy	Vercel, domain, analytics, Playwright journey tests for all four user journeys	16

Suggested first review checkpoint: end of Milestone 5. At that point there is a real desktop with draggable windows running placeholder content — enough to judge whether the feel is right before any content work is committed.

8. User journeys — acceptance criteria

Journey	Path	Passes when
Recruiter	Link → boot skip → Recruiter Mode → project → resume → contact	Name, title, top skills, best projects, education, resume and GitHub are all visible within 20 seconds of landing, on desktop <i>and</i> phone
Explorer	Boot → login → guest → desktop → Start → project → GitHub	Projects are reachable by at least four independent routes
Power user	Desktop → terminal → help → projects → open hotel-manager	Terminal can reach every application in the registry
Returning	Revisit → Quick Boot → desktop	Previous window layout and preferences restored in under a second

Journey	Path	Passes when
Keyboard only	Entire site, no mouse	Every application is reachable, operable and closable by keyboard
Shared deep link	/projects/hotel-manager pasted into LinkedIn	Correct title, description and preview image; page content readable without JavaScript

9. Standing principles

1. **One data source feeds every surface.** Duplicated content means a design error.
 2. **The creative layer never gates the professional layer.** Every piece of career information is reachable in under three interactions, and via a plain URL.
 3. **Applications are chrome-agnostic.** If an app component knows it is in a window, mobile is broken.
 4. **Honest over impressive.** Real statuses, real skill levels, real role descriptions. The metaphor is fiction; the content is not.
 5. **Accessibility is a constraint, not a milestone.** A component that cannot be operated by keyboard is not finished.
 6. **Every asset is originally created or properly licensed,** with provenance recorded.
 7. **Ship V1. Then improve it.** V2 ideas get written down, not built.
-

10. Immediate next steps

1. Seedorf answers the open items in §1.4
2. Milestone 0 — project foundation
3. Milestone 1 — content model, with placeholders where facts are pending
4. Milestone 2 — brand system, reviewed before proceeding